

INNOVATECH CHILE

Sistema de Gestión de Despachos

Informe de Evidencias - Despliegue Completo

Evaluación Parcial N°2 - Introducción a Herramientas DevOps (ISY1101)

Junio 2026

Índice

1. Resumen Ejecutivo
2. Arquitectura del Sistema
3. Evidencias del Despliegue
 - 3.1 GitHub Actions - Workflows en Verde
 - 3.2 Frontend - Aplicación Cargando
 - 3.3 API Ventas - Respuesta Correcta
 - 3.4 API Despachos - Respuesta Correcta
 - 3.5 EC2 - Instancias Ejecutándose
 - 3.6 EC2 - Contenedores en Ejecución (SSH)
 - 3.7 ECR - Repositorios con Imágenes
 - 3.8 GitHub Secrets Configurados
 - 3.9 Dockerfiles Multi-Stage
 - 3.10 docker-compose.yml - Stack Completo
 - 3.11 README.md - Documentación
 - 3.12 Git Log - Commits Descriptivos
4. Conclusión
5. Anexos

1. Resumen

El proyecto semestral de Innovatech Chile consiste en un sistema de gestión de despachos basado en microservicios. La aplicación cuenta con un frontend en React + Vite, dos backends en Spring Boot (ventas y despachos), y una base de datos MySQL. Todo el stack está containerizado con Docker, orquestado con Docker Compose y desplegado en AWS EC2 con automatización CI/CD mediante GitHub Actions.

Este informe recopila las evidencias del despliegue completo del sistema. Se incluyen capturas de pantalla que demuestran el funcionamiento de la contenerización, los pipelines CI/CD, las instancias EC2, los repositorios ECR, las APIs, y la integración Frontend-Backend.

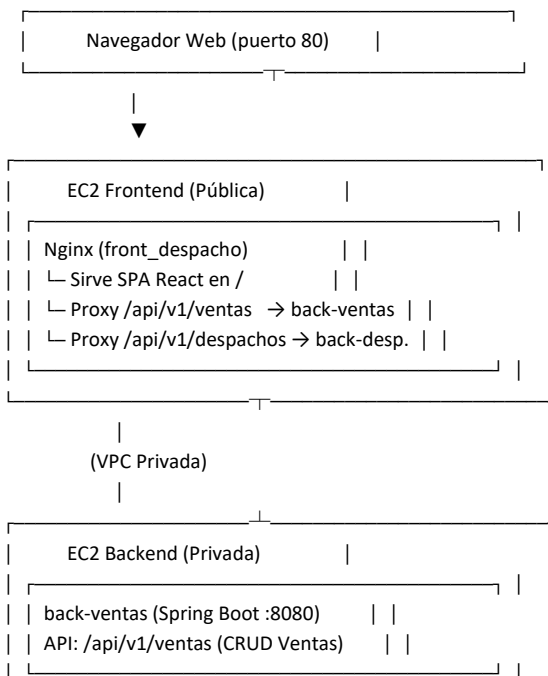
Datos del Proyecto:

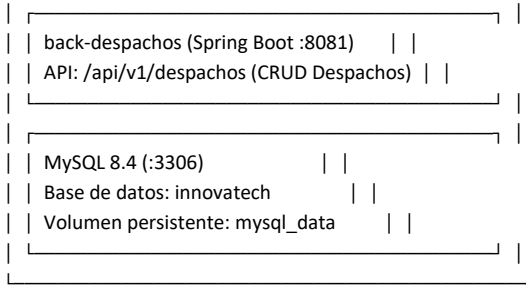
- Repositorio GitHub: <https://github.com/jsgiaverini/innovatech-devops-despacho>
- Rama de despliegue: deploy
- Frontend (público): <http://3.227.232.165>
- API Ventas: <http://3.227.232.165/api/v1/ventas>
- API Despachos: <http://3.227.232.165/api/v1/despachos>
- Registro ECR: Amazon ECR (front-despacho, back-ventas, back-despachos)
- Infraestructura: AWS EC2 (Frontend público + Backend privado)

2. Arquitectura del Sistema

El sistema sigue una arquitectura de microservicios con separación Frontend-Backend en dos instancias EC2 distintas: una pública para el frontend y una privada para los backends y la base de datos.

Diagrama de Arquitectura:





Componentes del Stack:

Componente	Tecnología	Puerto	Instancia EC2
Frontend	React + Vite + Nginx	80 (HTTP)	Frontend (Pública)
Backend Ventas	Spring Boot 3 + Java 17	8080	Backend (Privada)
Backend Despachos	Spring Boot 3 + Java 17	8081	Backend (Privada)
Base de Datos	MySQL 8.4	3306 (interno)	Backend (Privada)

Seguridad:

- Instancia Backend en subred privada sin IP pública
- MySQL (3306) no se expone externamente - comunicación interna por red Docker
- Security Groups: Frontend solo HTTP(80) y SSH(22); Backend solo TCP(8080/8081) desde Frontend SG
- Usuarios no root en todos los contenedores (principio de mínimo privilegio)
- SSH temporal desde GitHub Actions hacia EC2 (se abre y revoca en cada pipeline)

3. Evidencias del Despliegue

A continuación, se presentan las 12 evidencias que demuestran el despliegue completo del sistema Innovatech en AWS EC2 con automatización CI/CD.

3.1. GitHub Actions - Workflows en Verde

Fuente: [GitHub.com](#) > Repositorio > Actions

Los 3 pipelines CI/CD (Frontend, Backend Ventas, Backend Despachos) deben aparecer con check verde (success) en la pestaña Actions del repositorio:

<https://github.com/jsgiaverini/innovatech-devops-despacho/actions>

jsgiaverini / innovatech-devops-despacho

Q Type to search

<> Code

Issues

Pull requests

Actions

Projects

Wiki

Security and quality

Insights

Settings

Actions

New workflow

All workflows

Backend Despachos CI/CD - Build, Push ...

Backend Ventas CI/CD - Build, Push & D...

Frontend CI/CD - Build, Push & Deploy

Management

Caches

Attestations

Runners

Usage metrics

All workflows

Showing runs from all workflows

Q Filter workflow runs

12 workflow runs

Workflow

Event

Status

Branch

Actor

✓ Corriga build frontend y despliegue backend en workflows

Frontend CI/CD - Build, Push & Deploy #4: Commit 0563e61 pushed by jsgiaverini

deploy

39 minutes ago

44s

...

✓ Corriga build frontend y despliegue backend en workflows

Backend Despachos CI/CD - Build, Push & Deploy #4: Commit 0563e61 pushed by jsgiaverini

deploy

39 minutes ago

2m 41s

...

✓ Corriga build frontend y despliegue backend en workflows

Backend Ventas CI/CD - Build, Push & Deploy #4: Commit 0563e61 pushed by jsgiaverini

deploy

39 minutes ago

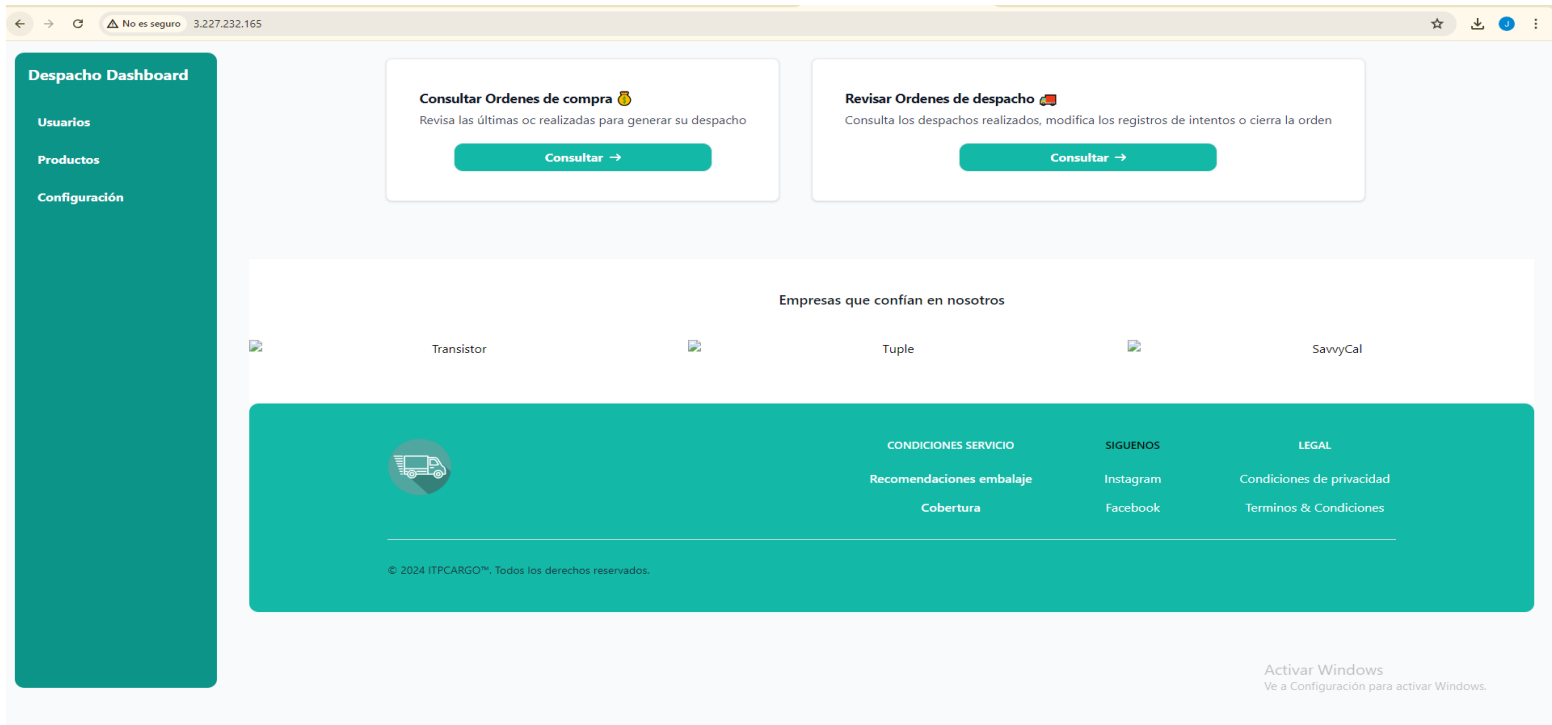
1m 12s

...

3.2. Frontend - Aplicación Cargando en Navegador

Fuente: Navegador web > <http://3.227.232.165>

Captura del navegador mostrando la aplicación React + Vite de Innovatech cargando correctamente en:
<http://3.227.232.165>

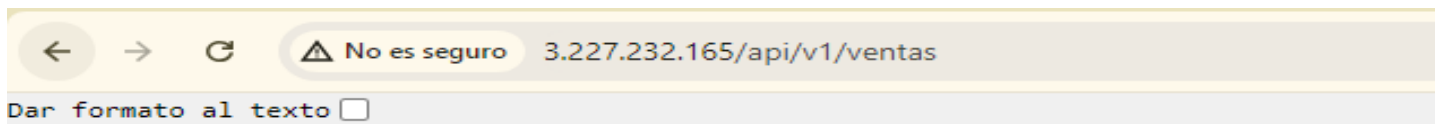


3.3. API Ventas - Respuesta Correcta

Fuente: Navegador web > <http://3.227.232.165/api/v1/ventas>

Captura del navegador o Postman mostrando la respuesta JSON de la API de ventas:
<http://3.227.232.165/api/v1/ventas>

Respuesta esperada: [] (arreglo vacío, indicando que la API responde correctamente)



[]

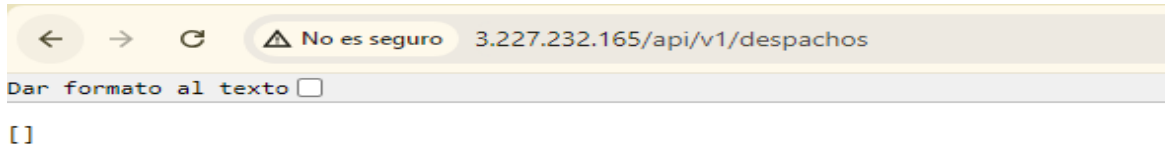
3.4. API Despachos - Respuesta Correcta

Fuente: Navegador web > <http://3.227.232.165/api/v1/despachos>

Captura del navegador o Postman mostrando la respuesta JSON de la API de despachos:

<http://3.227.232.165/api/v1/despachos>

Respuesta esperada: [] (arreglo vacío)

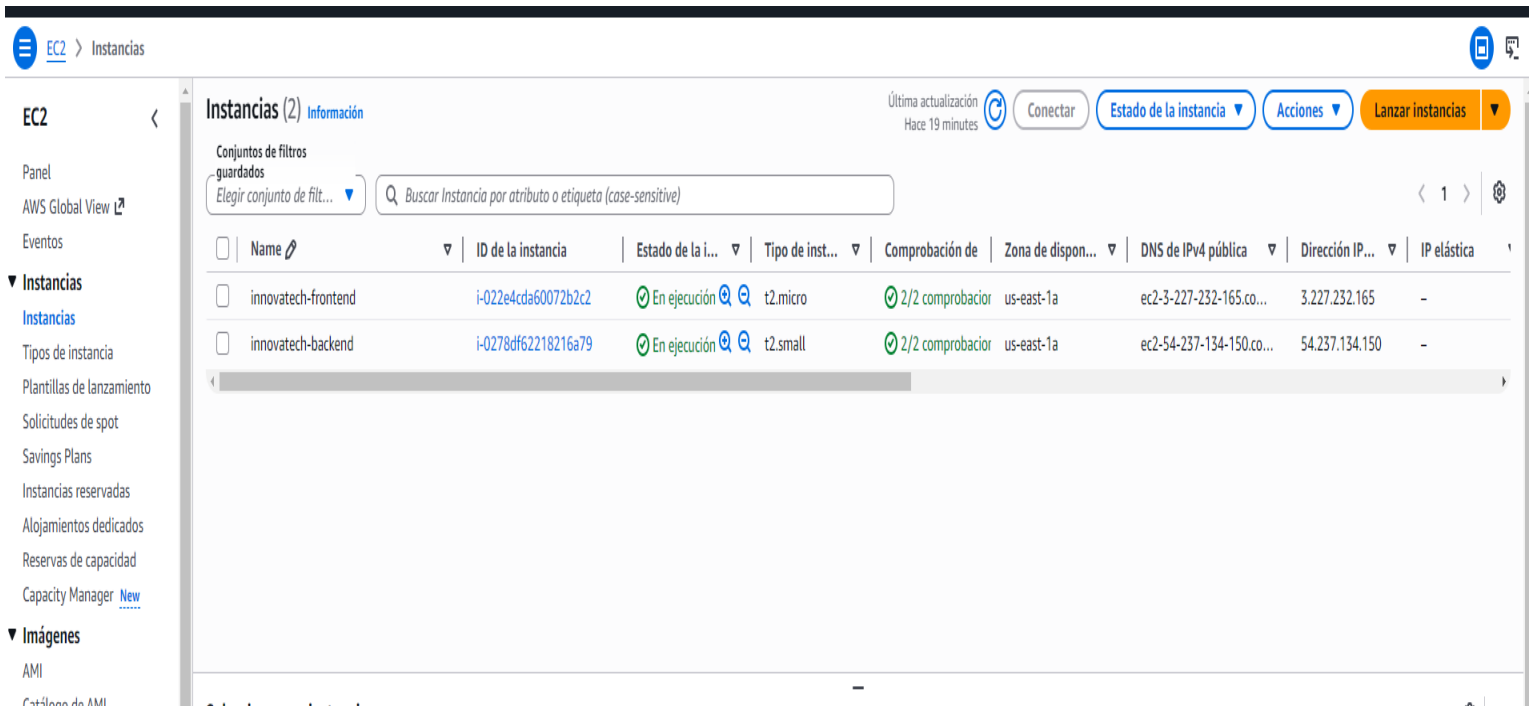


3.5. EC2 - Instancias Ejecutándose

Fuente: Consola AWS > EC2 > Instancias

Captura de la consola AWS > EC2 > Instances mostrando:

- innovatech-frontend (instancia pública) - Estado: Running
- innovatech-backend (instancia privada) - Estado: Running



3.6. EC2 - Contenedores en Ejecución (SSH)

Fuente: Terminal > SSH a EC2 > docker ps

Captura de la terminal SSH mostrando la salida de:

En EC2 Frontend: `docker ps` (debe mostrar `front_despacho` corriendo)

En EC2 Backend: `docker ps` (debe mostrar `innovatech_mysql`, `back_ventas`, `back_despachos`)

```
PS C:\Users\Julio\Desktop\Ingenieria Informatica Desarrollo de Software\5to Semestre - 2026\Introducción herramientas DevOps\proyecto semestral> $PEM_PATH="C:\Users\Julio\Desktop\DevOps\labsuser.pem"
>> $FRONT_PUBLIC_IP="3.227.232.165"
>> $BACK_PUBLIC_IP="54.237.134.158"
PS C:\Users\Julio\Desktop\Ingenieria Informatica Desarrollo de Software\5to Semestre - 2026\Introducción herramientas DevOps\proyecto semestral> ssh -i "$PEM_PATH" ec2-user@$FRONT_PUBLIC_IP

#
##### Amazon Linux 2023
#####
#####
#/ ____ https://aws.amazon.com/linux/amazon-linux-2023
Vv' ^->

Last login: Wed Jun 3 05:54:00 2026 from 181.42.141.147
[ec2-user@ip-172-31-14-58 ~]$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
047f4113d547   381491863278.dkr.ecr.us-east-1.amazonaws.com/front-despacho:latest "/docker-entrypoint..." 52 minutes ago Up 52 minutes (healthy) 0.0.0.0:80->8080/tcp, :::80->8080/tcp front_despacho
[ec2-user@ip-172-31-14-58 ~]$
```

```
C:\Users\Julio\Desktop>Ingenieria Informatica Desarrollo de Software\Sto Semestre - 2026\Introducción herramientas DevOps/proyecto semestral> ssh -i "C:\\$PROMPT_PATH\\ec2-user@back_public_ip" ec2-user@back_public_ip
```

```
#  
~\ #### Amazon Linux 2023  
  
#####  
#####  
###  
#/  
https://aws.amazon.com/linux/amazon-linux-2023  
Vw' '->  
  
/'  
/'  
/'  
/_m/'  

```

Last login: Wed Jun 3 05:34:28 2026 from 181.42.141.147

```
[ec2-user@ip-172-31-15-59 ~]$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a3cdd12c7d1e	381491863278.dkr.ecr.us-east-1.amazonaws.com/back-despachos:latest	"java -jar /app/app..."	52 minutes ago	Up 52 minutes (healthy)	0.0.0.0:8081->8081/tcp, :::8081->8081/tcp	back_despachos
a27376bc55da	381491863278.dkr.ecr.us-east-1.amazonaws.com/back-ventas:latest	"java -jar /app/app..."	53 minutes ago	Up 53 minutes (healthy)	0.0.0.0:8080->8080/tcp, :::8080->8080/tcp	back_ventas
7ee39042de49	mysql:8.4	"docker-entrypoint.s..."	About an hour ago	Up 54 minutes (healthy)	3306/tcp, 33060/tcp	innovatech_mysql

```
[ec2-user@ip-172-31-15-59 ~]$
```

You la semana pasada · isojaiverini (Hace 1 semana) Lin 13 col 36 Espacios: 4 UTF-8 LF Java Iniciar sesión Go Live Prettier

3.7. ECR - Repositorios con Imágenes

Fuente: Consola AWS > ECR > Repositories

Captura de la consola AWS > ECR > Repositories mostrando los 3 repositorios:

- front-despacho (con imagen latest)
- back-ventas (con imagen latest)
- back-despachos (con imagen latest)

Amazon ECR

Registro privado

Repositorios

Amazon Elastic Container Registry

▼ Registro privado

Repositorios

Características y ajustes

▼ Registro público

Repositorios

Configuración

Galería pública de ECR

Amazon ECS

Amazon EKS

Empezar a utilizar

Documentación

Repositorios privados (3)

Buscar por subcadena de repositorio

	Nombre del repositorio	URI	Creado en	Inmutabilidad de etiqueta	Tipo de cifrado
☐	back-despachos	381491863278.dkr.ecr.us-east-1.amazonaws.com/back-despachos	27 de mayo de 2026, 01:58:37 (UTC-04)	Mutable	AES-256
☐	back-ventas	381491863278.dkr.ecr.us-east-1.amazonaws.com/back-ventas	27 de mayo de 2026, 01:58:30 (UTC-04)	Mutable	AES-256
☐	front-despacho	381491863278.dkr.ecr.us-east-1.amazonaws.com/front-despacho	27 de mayo de 2026, 01:58:23 (UTC-04)	Mutable	AES-256

Activar Windows
Ve a Configuración para activar Windows.

back-despachos

Resumen

Imágenes

Política de ciclo de vida

Permisos

Etiquetas de repositorio

Imágenes (6)

Información

Eliminar

Copiar URI

Detalles

Analizar

Ver comandos de envío

Filtrar imágenes activas

	Etiquetas de imagen	Tipo	Creada a las	Tamaño de imagen (MB)	Resumen de imagen	Extraída por última vez a las
☐	latest	Image	03 de junio de 2026, 03:17:01 (UTC-04)	123.23	sha256:e03238e4e9cbed...	03 de junio de 2026, 03:17:08 (UTC-04)
☐	-	Image	03 de junio de 2026, 03:06:11 (UTC-04)	123.23	sha256:fce59c6a09c240e...	03 de junio de 2026, 03:06:12 (UTC-04)
☐	-	Image	03 de junio de 2026, 02:43:19 (UTC-04)	123.23	sha256:3d0cde3f47bfcf11...	03 de junio de 2026, 02:43:26 (UTC-04)
☐	-	Image Index	27 de mayo de 2026, 02:01:19 (UTC-04)	178.51	sha256:a3e44b1074e013...	03 de junio de 2026, 02:42:50 (UTC-04)
☐	-	Image	27 de mayo de 2026, 02:01:19 (UTC-04)	178.51	sha256:66841ab9b64519...	27 de mayo de 2026, 23:05:03 (UTC-04)
☐	-	Image	27 de mayo de 2026, 02:01:19 (UTC-04)	0.00	sha256:d84ac6401bd5f8a...	-

3.8. GitHub Secrets Configurados

Fuente: GitHub > Settings > Secrets and variables > Actions

Captura de GitHub > Settings > Secrets and variables > Actions mostrando los secrets configurados (sin revelar valores):

AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN, AWS_REGION, ECR_REGISTRY, EC2_FRONTEND_HOST, EC2_BACKEND_HOST, BACKEND_PRIVATE_IP, EC2_SSH_KEY, FRONTEND_SG_ID, BACKEND_SG_ID, DB_NAME, DB_USERNAME, DB_PASSWORD, MYSQL_ROOT_PASSWORD

The screenshot shows the GitHub Settings page for the repository 'innovatech-devops-despacho'. The left sidebar contains navigation options: General, Access, Collaborators, Moderation options, Code and automation (Branches, Tags, Rules, Actions, Models, Webhooks, Copilot, Environments, Codespaces, Pages), Security and quality (Advanced Security, Deploy keys, Secrets and variables), and Integrations (Agents, Codespaces, Dependabot). The 'Secrets and variables' section is expanded, showing 'Actions' as the selected tab. The main content area is titled 'Actions secrets and variables' and includes a description of secrets and variables, tabs for 'Secrets' and 'Variables', and a section for 'Environment secrets' which currently shows 'This environment has no secrets.' Below this is a 'Repository secrets' section with a 'New repository secret' button and a table of configured secrets.

Name	Last updated
AWS_ACCESS_KEY_ID	1 hour ago
AWS_REGION	1 hour ago
AWS_SECRET_ACCESS_KEY	1 hour ago
AWS_SESSION_TOKEN	1 hour ago
BACKEND_PRIVATE_IP	1 hour ago
BACKEND_SG_ID	1 hour ago

This screenshot provides a closer view of the 'Repository secrets' section in the GitHub Settings. The left sidebar is partially visible, showing 'Webhooks', 'Copilot', 'Environments', 'Codespaces', 'Pages', 'Security and quality', 'Advanced Security', 'Deploy keys', 'Secrets and variables', and 'Integrations'. The 'Secrets and variables' section is expanded, with 'Actions' selected. The 'Repository secrets' table lists 15 secrets, each with a 'Name' column and a 'Last updated' column. Each row includes a lock icon, the secret name, the update time ('1 hour ago'), and edit/delete icons.

Name	Last updated
AWS_ACCESS_KEY_ID	1 hour ago
AWS_REGION	1 hour ago
AWS_SECRET_ACCESS_KEY	1 hour ago
AWS_SESSION_TOKEN	1 hour ago
BACKEND_PRIVATE_IP	1 hour ago
BACKEND_SG_ID	1 hour ago
DB_NAME	1 hour ago
DB_PASSWORD	1 hour ago
DB_USERNAME	1 hour ago
EC2_BACKEND_HOST	1 hour ago
EC2_FRONTEND_HOST	1 hour ago
EC2_SSH_KEY	1 hour ago
FRONTEND_SG_ID	1 hour ago
MYSQL_ROOT_PASSWORD	1 hour ago

3.9. Dockerfiles Multi-Stage

Fuente: Repositorio GitHub > Archivos Dockerfile

Captura del código de los 3 Dockerfiles mostrando:

- front_despacho/Dockerfile: build con node:24-alpine, final con nginx no root, HEALTHCHECK
- back-Ventas/Dockerfile: build con maven, final con temurin JRE, usuario spring no root, HEALTHCHECK
- back-Despachos/Dockerfile: igual al de ventas, puerto 8081

innovatech-devops-despacho / front_despacho / Dockerfile

jsgiaverini Corrige build frontend y despliegue backend en workflows

0563e61 · 1 hour ago History

Code Blame 24 lines (13 loc) · 503 Bytes

Raw Download Edit

```
1 FROM node:24-alpine AS build
2
3 WORKDIR /app
4
5 COPY package*.json ./
6
7 RUN if [ -f package-lock.json ]; then npm ci; else npm install; fi
8
9 COPY . .
10
11 RUN npm run build
12
13 FROM nginxinc/nginx-unprivileged:1.27-alpine
14
15 COPY nginx.conf.template /etc/nginx/templates/default.conf.template
16
17 COPY --from=build /app/dist /usr/share/nginx/html
18
19 EXPOSE 8080
20
21 HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
22   CMD wget -qO- http://localhost:8080/ || exit 1
23
24 CMD ["nginx", "-g", "daemon off;"]
```

Activar Windows
Ve a Configuración para activar Windows.

innovatech-devops-despacho / back-Despachos_SpringBoot / Springboot-API-REST-DESPACHO / Dockerfile

jsgiaverini Actualiza CI/CD, documentacion y configuracion de despliegue AWS

Code Blame 26 lines (15 loc) · 610 Bytes

Raw Download Edit

```
1 FROM maven:3.9-eclipse-temurin-17-alpine AS build
2
3 WORKDIR /app
4
5 COPY pom.xml .
6 RUN mvn -q -DskipTests dependency:go-offline
7
8 COPY src ./src
9 RUN mvn -q clean package -DskipTests
10
11 FROM eclipse-temurin:17-jre-alpine
12
13 RUN addgroup -S spring && adduser -S spring -G spring
14
15 WORKDIR /app
16
17 COPY --from=build --chown=spring:spring /app/target/*.jar app.jar
18
19 USER spring
20
21 EXPOSE 8081
22
23 HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
24   CMD wget -qO- http://localhost:8081/actuator/health || wget -qO- http://localhost:8081/swagger-ui.html || exit 1
25
26 ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

Activar Win

```

1 FROM maven:3.9-eclipse-temurin-17-alpine AS build
2
3 WORKDIR /app
4
5 COPY pom.xml .
6 RUN mvn -q -DskipTests dependency:go-offline
7
8 COPY src ./src
9 RUN mvn -q clean package -DskipTests
10
11 FROM eclipse-temurin:17-jre-alpine
12
13 RUN addgroup -S spring && adduser -S spring -G spring
14
15 WORKDIR /app
16
17 COPY --from=build --chown=spring:spring /app/target/*.jar app.jar
18
19 USER spring
20
21 EXPOSE 8080
22
23 HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
24     CMD wget -qO- http://localhost:8080/actuator/health || wget -qO- http://localhost:8080/swagger-ui.html || exit 1
25
26 ENTRYPOINT ["java", "-jar", "/app/app.jar"]
    
```

Activar
Ve a Confi

3.10. docker-compose.yml - Stack Completo

Fuente: Repositorio GitHub > docker-compose.yml

Captura del archivo docker-compose.yml raíz mostrando:

- 4 servicios: mysql, back-ventas, back-despachos, frontend
- Red interna innovatech_net (bridge)
- Named volume mysql_data para persistencia
- Dependencias con condition: service_healthy
- Variables de entorno mapeadas desde .env

```

1 services:
2   mysql:
3     image: mysql:8.4
4     container_name: innovatech_mysql
5     restart: unless-stopped
6     environment:
7       MYSQL_DATABASE: ${DB_NAME}
8       MYSQL_USER: ${DB_USERNAME}
9       MYSQL_PASSWORD: ${DB_PASSWORD}
10      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
11     volumes:
12       # Named volume para persistencia de datos MySQL.
13       # Elegido sobre bind mount porque:
14       # 1. Docker gestiona el ciclo de vida del volumen (portabilidad)
15       # 2. Funciona igual en Windows, macOS y Linux
16       # 3. Los datos sobreviven a docker compose down (no así el contenedor)
17       # 4. Facilita backups con docker run --volumes-from
18       - mysql_data:/var/lib/mysql
19     networks:
20       - innovatech_net
21     healthcheck:
22       test: ["CMD-SHELL", "mysqladmin ping -h localhost -uroot -p${MYSQL_ROOT_PASSWORD}"]
23       interval: 10s
24       timeout: 5s
25       retries: 10
26       start_period: 30s
27
28   back-ventas:
29     build:
30       context: ../back-Ventas_SpringBoot/Springboot-API-REST
31       dockerfile: Dockerfile
32     image: back-ventas:local
33     container_name: back_ventas
    
```

Activar Windows
Ve a Configuración para activar W

Code Blame 109 lines (104 loc) • 3.46 KB

```

41 SPRING_DATASOURCE_URL: "jdbc:mysql://${DB_ENDPOINT}/${DB_PORT}/${DB_NAME}?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC&createDatabaseIfNotExist=true"
42 SPRING_DATASOURCE_USERNAME: ${DB_USERNAME}
43 SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
44 depends_on:
45   mysql:
46     condition: service_healthy
47 ports:
48   - "8080:8080"
49 networks:
50   - innovatech_net
51
52 back-despachos:
53   build:
54     context: ./back-Despachos_SpringBoot/Springboot-API-REST-DESPACHO
55     dockerfile: Dockerfile
56   image: back-despachos:local
57   container_name: back_despachos
58   restart: unless-stopped
59   environment:
60     DB_ENDPOINT: ${DB_ENDPOINT}
61     DB_PORT: ${DB_PORT}
62     DB_NAME: ${DB_NAME}
63     DB_USERNAME: ${DB_USERNAME}
64     DB_PASSWORD: ${DB_PASSWORD}
65     SPRING_DATASOURCE_URL: "jdbc:mysql://${DB_ENDPOINT}:${DB_PORT}/${DB_NAME}?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC&createDatabaseIfNotExist=true"
66     SPRING_DATASOURCE_USERNAME: ${DB_USERNAME}
67     SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
68   depends_on:
69     mysql:
70       condition: service_healthy
71   ports:
72     - "8081:8081"
73   networks:
74     - innovatech_net
75
76 frontend:
77   build:
78     context: ./front_despacho
79     dockerfile: Dockerfile
80   image: front-despacho:local
81   container_name: front_despacho
82   restart: unless-stopped
83   environment:
84     # Nainx envsubst reemplaza estos valores en el template al iniciar

```

Activar
Ve a Con

3.11. README.md - Documentación Completa

Fuente: Repositorio GitHub > README.md

Captura del README.md mostrando:

- Diagrama de arquitectura
- Estructura del proyecto
- Instrucciones de ejecución local
- Guía de despliegue AWS EC2
- Justificaciones técnicas (multi-stage, persistencia, ECR vs Docker Hub)
- Principios DevOps aplicados
- Consideraciones de seguridad

innovatech-devops-despacho / README.md

jsgiaverini Actualiza CI/CD, documentacion y configuracion de despliegue AWS

9d20a93 · yesterday

Preview Code Blame 437 lines (342 loc) · 18.1 KB

Raw Download Edit

Innovatech - Sistema de Gestión de Despachos

Aplicación empresarial para la gestión de órdenes de compra y despachos de la empresa **Innovatech Chile**. Sistema basado en microservicios con frontend React, backend Spring Boot, base de datos MySQL y despliegue automatizado en AWS EC2 mediante Docker y GitHub Actions CI/CD.

Arquitectura del Sistema



Activar Windows
Ve a Configuración para activar Window

innovatech-devops-despacho / README.md

Preview Code Blame 437 lines (342 loc) · 18.1 KB

Raw Download Edit

Estructura del Proyecto

```
proyecto_semestral/
├── .github/workflows/           # Pipelines CI/CD
│   ├── frontend-deploy.yml
│   ├── backend-ventas-deploy.yml
│   └── backend-despachos-deploy.yml
├── deploy/                     # Archivos de despliegue para EC2
│   ├── backend/
│   │   ├── docker-compose.yml
│   │   └── .env.example
│   ├── frontend/
│   │   ├── docker-compose.yml
│   │   └── .env.example
│   ├── back-Ventas_SpringBoot/
│   │   ├── Springboot-API-REST/
│   │   │   ├── Dockerfile
│   │   │   ├── pom.xml
│   │   │   └── src/
│   │   └── # Multi-stage, usuario no root
│   ├── back-Despachos_SpringBoot/
│   │   ├── Springboot-API-REST-DESPACHO/
│   │   │   ├── Dockerfile
│   │   │   ├── pom.xml
│   │   │   └── src/
│   │   └── # Multi-stage, usuario no root
│   └── front_despacho/
│       ├── Dockerfile
│       ├── nginx.conf.template
│       └── # Proxy reverso con env vars
├── docker-compose.yml          # Orquestador local
├── .env
├── .env.example
├── .gitignore
└── README.md
```

Activar Windows
Ve a Configuración para activar Window

Requisitos

3.12. Git Log - Commits Descriptivos

Fuente: Terminal > `git log --oneline` (o *GitHub Insights*)

Captura del historial de commits (`git log --oneline`) mostrando:

7 commits descriptivos con convención semántica:

- Corrige build frontend y despliegue backend en workflows
- Corrige rutas EC2 y espera de MySQL en workflows
- Mejora workflows con apertura temporal SSH para GitHub Actions
- Actualiza CI/CD, documentación y configuración de despliegue AWS
- Corrige compose backend para despliegue en EC2
- Agrega archivos compose para despliegue en EC2 usando ECR
- Configura contenedorización Docker para frontend y backends

```
21 @builder
22 public class Venta {
23     ...

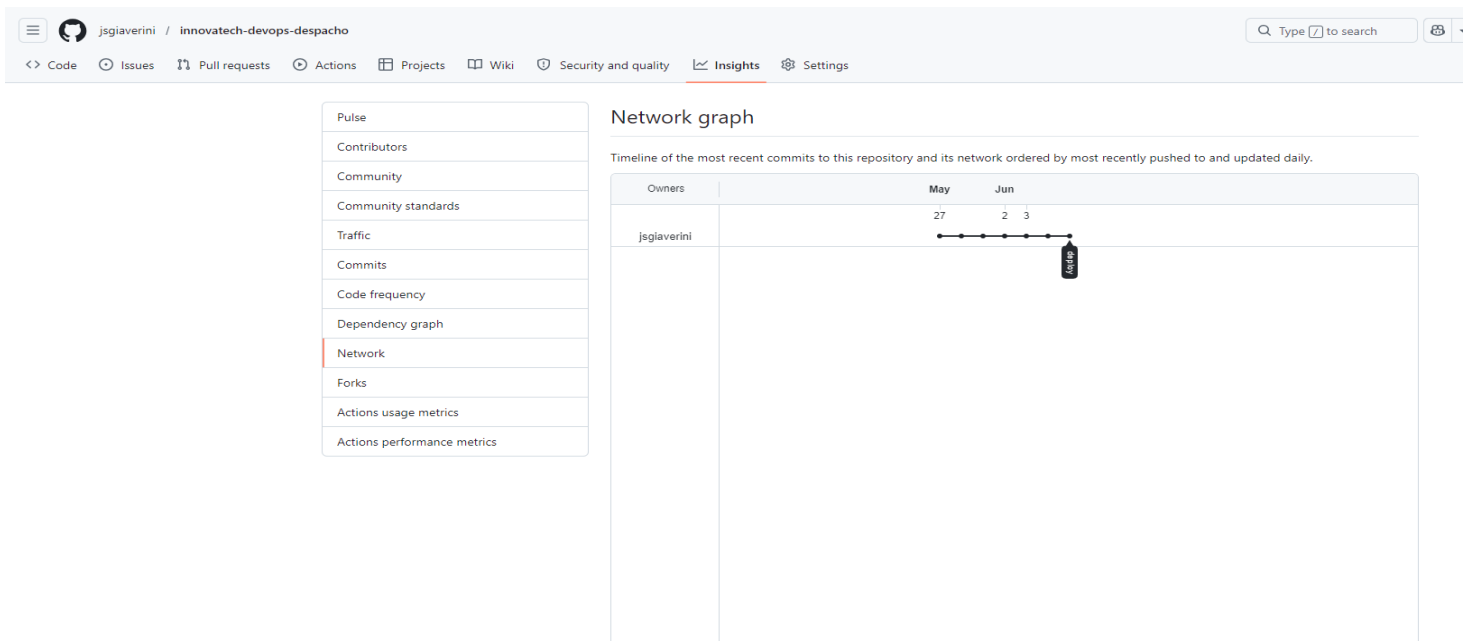
```

PROBLEMAS 6 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS GITLENS

PS C:\Users\Julio\Desktop\Ingeniería Informática Desarrollo de Software\5to Semestre - 2026\Introducción herramientas DevOps\proyecto semestral> `git log --oneline -10`

```
0563e61 (HEAD -> deploy, origin/deploy) Corrige build frontend y despliegue backend en workflows
5f149b9 Corrige rutas EC2 y espera de MySQL en workflows
c08506e Mejora workflows con apertura temporal SSH para GitHub Actions
9d20a93 Actualiza CI/CD, documentación y configuración de despliegue AWS
af59d95 Corrige compose backend para despliegue en EC2
cb15040 Agrega archivos compose para despliegue en EC2 usando ECR
03efcb5 Configura contenedorización Docker para frontend y backends
PS C:\Users\Julio\Desktop\Ingeniería Informática Desarrollo de Software\5to Semestre - 2026\Introducción herramientas DevOps\proyecto semestral>
```

Activar Windows
Ve a Configuración para activar Windows.



4. Conclusión

El proyecto Innovatech Chile - Sistema de Gestión de Despachos ha sido desplegado exitosamente en AWS EC2 cumpliendo con todos los indicadores de la rúbrica EP2:

- Contenedorización: 3 Dockerfiles multi-stage con usuario no root y HEALTHCHECK
- Orquestación: docker-compose.yml con 4 servicios, red interna y named volume
- CI/CD: 3 pipelines GitHub Actions completamente funcionales
- Frontend: Accesible públicamente en <http://3.227.232.165>
- Backend: APIs respondiendo correctamente con persistencia
- Documentación: README completo, commits descriptivos

El flujo completo de CI/CD funciona correctamente: el push a la rama deploy dispara la construcción de la imagen Docker, su publicación en Amazon ECR, y el despliegue automatizado en EC2 mediante SSH temporal, demostrando la aplicación práctica de principios DevOps como contenedorización, integración continua, despliegue continuo, y gestión de configuración.

5. Anexos

A.1 - Dockerfile Frontend:

```
FROM node:24-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN if [ -f package-lock.json ]; then npm ci; else npm install; fi
COPY . .
RUN npm run build

FROM nginxinc/nginx-unprivileged:1.27-alpine
COPY nginx.conf.template /etc/nginx/templates/default.conf.template
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget -qO- http://localhost:8080/ || exit 1
CMD ["nginx", "-g", "daemon off;"]
```

A.2 - Dockerfile Backend (Ventas/Despachos):

```
FROM maven:3.9-eclipse-temurin-17-alpine AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -q -DskipTests dependency:go-offline
COPY src ./src
RUN mvn -q clean package -DskipTests

FROM eclipse-temurin:17-jre-alpine
RUN addgroup -S spring && adduser -S spring -G spring
WORKDIR /app
COPY --from=build --chown=spring:spring /app/target/*.jar app.jar
USER spring
EXPOSE 8080 # o 8081 para despachos
HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
  CMD wget -qO- http://localhost:8080/actuator/health || ... || exit 1
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

A.3 - Variables de Entorno (.env.example):

```
DB_ENDPOINT=mysql
DB_PORT=3306
DB_NAME=innovatech
DB_USERNAME=appuser
DB_PASSWORD=change_me
MYSQL_ROOT_PASSWORD=change_me
VENTAS_HOST=back-ventas
DESPACHOS_HOST=back-despachos
```